Systemy rekomendacyjne ogłoszeń OLX
Raport z eksperymentów

Grupa 5

21 października 2026

Streszczenie

Sprawdzono trzy sposoby polecania ogłoszeń: wybór najpopularniejszych ofert (Popularity), wykorzystanie kolejności kliknięć (MCRS) oraz model neuronowy SASRec. Porównano jakość rekomendacji i czas obliczeń. Każdą metodę oceniono zarówno z pomijaniem ogłoszeń wcześniej klikniętych przez użytkownika, jak i z możliwością ponownego ich polecenia. Dodatkowo sprawdzono, jak wyniki zmieniają się przy użyciu 10%, 50% i 100% dostępnych danych.

1 Dane i podział

Wykorzystano 2 453 578 kliknięć dotyczących 13 317 ogłoszeń pochodzących od 239 252 użytkowników. Model uczył się na 2 076 316 kliknięciach, a do sprawdzenia wyników użyto 377 262 późniejszych kliknięć. Oceniono 44 316 użytkowników obecnych w obu częściach danych.

Skrypt przygotowania danych losuje 10% użytkowników (ziarno losowe 42) i zachowuje 10% ogłoszeń najpopularniejszych w części treningowej. Nie oznacza to wykorzystania dokładnie 10% wszystkich kliknięć OLX.

Dane podzielono według czasu: wcześniejsze kliknięcia służą do nauki, a późniejsze do oceny. Granicę wyznaczono po pierwszych 80% kliknięć wylosowanych użytkowników, przed ograniczeniem listy ogłoszeń. Po tym ograniczeniu proporcja liczby kliknięć w obu częściach może być inna.

2 Metody i konfiguracja

Popularity: metoda referencyjna. Metoda liczy kliknięcia każdego ogłoszenia w danych treningowych i poleca dziesięć najpopularniejszych dozwolonych ofert. Jest punktem odniesienia: pozwala sprawdzić, czy bardziej złożone algorytmy dają lepsze wyniki.

MCRS: łańcuch Markowa. Metoda sprawdza, które ogłoszenia użytkownicy klikali bezpośrednio po sobie. Na tej podstawie szacuje prawdopodobieństwo przejścia z ogłoszenia i do ogłoszenia j:

$$P(j \mid i) = \frac{N_{ij}}{\sum_k N_{ik}},$$

gdzie N_{ij} oznacza liczbę takich przejść w danych treningowych. Rekomendacje zależą od ostatniego ogłoszenia klikniętego przez użytkownika. Jeśli nie ma danych o przejściach, metoda korzysta z popularności ogłoszeń. Gdy zaobserwowanych przejść jest za mało, obecny kod może również wybrać ogłoszenia z zerowym prawdopodobieństwem. Ich kolejność przy remisie może wpływać na wynik.

SASRec: model neuronowy. Model analizuje historię kliknięć i uczy się przewidywać kolejne ogłoszenie. Wykorzystuje Transformer, który pozwala uwzględniać różne elementy historii. Podczas nauki dla każdej pozycji widzi tylko wcześniejsze i bieżące kliknięcia. Ogłoszenia i ich pozycje w historii są zapisywane jako uczone wektory liczb. Model uczy się odróżniać rzeczywiście kliknięte ogłoszenie od losowo wybranego, którego użytkownik nie kliknął w treningu. Użyto binarnej entropii krzyżowej osobno dla obu przykładów. Implementacja w PyTorch jest inspirowana SASRec i nie odtwarza dokładnie wszystkich szczegółów kodu autorów. Jej ustawienia przedstawiono poniżej.

Parametr	Wartość
Liczba przejść przez dane (epoki)	10
Liczba historii przetwarzanych jednocześnie	512
Maksymalna długość sekwencji	50
Długość wektora opisującego ogłoszenie	64
Bloki Transformera / głowy uwagi	2 / 1
Dropout	0,2
Optymalizator	Adam
Krok uczenia	0,001
Parametry Adam β_1, β_2	0,9; 0,98
Ziarno losowe	42

3 Procedura ewaluacji

Modele korzystają wyłącznie z danych treningowych. Dla każdego ocenianego użytkownika tworzą listę do dziesięciu ogłoszeń. Trafieniem jest ogłoszenie, które znalazło się na tej liście i zostało kliknięte przez użytkownika w teście. Użyto dwóch miar:

- **Precision@10**: liczba trafień podzielona przez 10. Opisuje, jaka część listy rekomendacji była trafna.
- **Recall@10**: liczba trafień podzielona przez liczbę różnych ogłoszeń klikniętych przez użytkownika w teście. Opisuje, jak dużą część jego późniejszych wyborów udało się przewidzieć.

Obie miary policzono osobno dla każdego użytkownika, a następnie uśredniono. Wielokrotne kliknięcie tego samego ogłoszenia jest traktowane jako jeden wybór. W Precision@10 mianownik zawsze wynosi 10, również przy krótszej liście. W tabelach użyto skrótów P@10 i R@10.

Każdą metodę sprawdzono w dwóch wariantach:

- **Z filtrowaniem**: pomijane są ogłoszenia, które użytkownik kliknął już w danych treningowych.
- **Bez filtrowania**: można ponownie polecić wcześniej kliknięte ogłoszenie.

W obu wariantach trafienia porównuje się z tym samym zbiorem testowym. Jeżeli użytkownik wrócił w teście do znanego ogłoszenia, wariant z filtrowaniem nie może go trafić. Dlatego wyższy wynik bez filtrowania nie musi oznaczać lepszego polecania nowych ofert.

4 Wyniki eksperymentu AWS

Metoda	Filtr	P@10	R@10	Trening [s]	Ocena [s]	Pred. [ms]
Popularity	tak	0,001374	0,002456	4,14	0,162	0,00246
Popularity	nie	0,001902	0,003152	4,14	0,109	0,00155
MCRS	tak	0,018797	0,055447	6,28	41,015	0,91117
MCRS	nie	0,041475	0,131978	6,28	40,664	0,86795
SASRec	tak	0,025939	0,061299	1452,87	12,275	3,36977
SASRec	nie	0,056135	0,154968	1452,87	11,014	3,06514

Kolumna **Trening** podaje czas przygotowania i uczenia modelu, **Ocena** czas wygenerowania rekomendacji i obliczenia metryk dla wszystkich ocenianych użytkowników, a **Pred.** średni czas jednej rekomendacji dla jednego użytkownika. Ten ostatni pomiar wykonano na 100 użytkownikach, po wstępnym wywołaniu modelu.

Podczas pełnej oceny SASRec obsługuje po 512 użytkowników jednocześnie, a pozostałe metody po jednym. Krótszy czas całej oceny SASRec nie oznacza więc krótszego czasu pojedynczego zapytania.

Dla MCRS czas treningu obejmuje zliczenie przejść, przygotowanie historii i rankingu popularności. Dla Popularity jest to przygotowanie historii i rankingu, a dla SASRec przygotowanie historii oraz dziesięć epok nauki. Pobierania i wczytywania plików CSV nie uwzględniono. Model uczono raz, a następnie oceniano w obu wariantach filtrowania. Dlatego w obu wierszach danej metody występuje ten sam czas treningu.

Sprzęt AWS. Eksperyment wykonano w AWS SageMaker JupyterLab na CPU, bez GPU. Środowisko udostępniało 4 procesory logiczne i około 15,42 GiB RAM (16 552 808 448 bajtów). Użyto systemu Linux x86_64, Pythona 3.12.14 i PyTorch 2.8.0. Nie zapisano dokładnego modelu procesora ani typu instancji. Podana pamięć RAM oznacza wielkość pamięci widocznej w systemie, a nie maksymalne zużycie podczas obliczeń.

5 Wpływ wielkości danych

Dodatkowe eksperymenty wykonano dla 10%, 50% i 100% użytkowników z dostępnych plików treningowych i testowych. Losowanie z ziarnem 42 zapewnia, że mniejsza próbka zawiera się w większej. Dla wybranych użytkowników zachowano wszystkie kliknięcia i ten sam podział według czasu. Nie ograniczano ponownie listy ogłoszeń do najpopularniejszych ofert.

Te procenty odnoszą się do dostępnego, wcześniej przygotowanego zbioru, a nie do wszystkich danych OLX. Między próbkami zmienia się zarówno ilość danych do nauki, jak i grupa ocenianych użytkowników. Wyniki pokazują zatem łączny efekt tych zmian.

5.1 Eksperymenty lokalne

Wszystkie poniższe pomiary wykonano na tym samym komputerze: macOS ARM64, 10 procesorów logicznych, 24 GiB RAM, CPU bez GPU; PyTorch 2.14.1, Python 3.13.14. PyTorch ograniczono do 4 wątków. Próbki zawierają kolejno 23 925, 119 626 oraz 239 252 użytkowników. Nie należy bezpośrednio porównywać tych czasów z pomiarami AWS.

Próbka	Użytkownicy	Ogłoszenia	Train	Test	Ocena (użytk.)
10%	23925	13315	205183	37448	4392
50%	119626	13317	1036622	188039	22273
100%	239252	13317	2076316	377262	44316

Z filtrowaniem.

Próbka	Metoda	P@10	R@10	Trening [s]	Ocena [s]	Pred. [ms]
10%	Popularity	0,001457	0,003138	0,09	0,003	0,0005
10%	MCRS	0,008948	0,027145	0,11	0,346	0,0783
10%	SASRec	0,010405	0,023837	23,02	0,190	0,3801
50%	Popularity	0,001289	0,002265	0,48	0,020	0,0006
50%	MCRS	0,015476	0,044965	0,65	1,812	0,0815
50%	SASRec	0,022350	0,053551	113,32	1,000	0,3781
100%	Popularity	0,001374	0,002456	1,01	0,040	0,0006
100%	MCRS	0,018842	0,055157	1,42	3,653	0,0804
100%	SASRec	0,025943	0,061639	228,24	1,859	0,3732

Bez filtrowania.

Próbka	Metoda	P@10	R@10	Trening [s]	Ocena [s]	Pred. [ms]
10%	Popularity	0,002140	0,004099	0,09	0,002	0,0003
10%	MCRS	0,024772	0,079533	0,11	0,342	0,0780
10%	SASRec	0,025797	0,062774	23,02	0,153	0,3665
50%	Popularity	0,001859	0,002979	0,48	0,012	0,0003
50%	MCRS	0,037022	0,117443	0,65	1,791	0,0787
50%	SASRec	0,050366	0,139975	113,32	0,797	0,3704
100%	Popularity	0,001902	0,003152	1,01	0,026	0,0004
100%	MCRS	0,041651	0,132182	1,42	3,635	0,0816
100%	SASRec	0,055824	0,154303	228,24	1,704	0,4580

Obserwacje dotyczące wielkości próbki. Popularity, z filtrowaniem: dla 10% i 100% próbki Precision@10 wynosi odpowiednio 0,001457 i 0,001374, a Recall@10 0,003138 i 0,002456. Popularity, bez filtrowania: dla 10% i 100% próbki Precision@10 wynosi odpowiednio 0,002140 i 0,001902, a Recall@10 0,004099 i 0,003152. MCRS, z filtrowaniem: dla 10% i 100% próbki Precision@10 wynosi odpowiednio 0,008948 i 0,018842, a Recall@10 0,027145 i 0,055157. MCRS, bez filtrowania: dla 10% i 100% próbki Precision@10 wynosi odpowiednio 0,024772 i 0,041651, a Recall@10 0,079533 i 0,132182. SASRec, z filtrowaniem: dla 10% i 100% próbki Precision@10 wynosi odpowiednio 0,010405 i 0,025943, a Recall@10 0,023837 i 0,061639. SASRec, bez filtrowania: dla 10% i 100% próbki Precision@10 wynosi odpowiednio 0,025797 i 0,055824, a Recall@10 0,062774 i 0,154303. Wzrost populacji zwiększa liczbę dostępnych przejść i sekwencji, ale także koszt obliczeń. Wnioski liczbowe dotyczą tego konkretnego losowania i nie stanowią dowodu monotonicznej zależności dla dowolnej próbki.

Jakość a koszt. Dla 10% próbki SASRec ma wyższe Precision@10, ale niższe Recall@10 niż MCRS w obu wariantach. Dla 50% i 100% wygrywa w obu metrykach. Trening SASRec trwa kolejno 23,02 s, 113,32 s i 228,24 s, co w tych pomiarach odpowiada wzrostowi kosztu zbliżonemu do wzrostu liczby interakcji. Dla MCRS są to odpowiednio 0,11 s, 0,65 s i 1,42 s. Baseline nie wykazuje monotonicznej poprawy jakości wraz z rozmiarem próbki. Drobne różnice jakości między przebiegami AWS i lokalnym przy 100% są możliwe mimo wspólnego ziarna: środowiska mają inne wersje bibliotek, a remisy rankingów i obliczenia numeryczne nie muszą dawać identycznych wyników.

6 Wnioski i ograniczenia

W eksperymencie AWS najlepszą jakość uzyskał SASRec. Z filtrowaniem jego Precision@10 było wyższe od MCRS o około 38%, a Recall@10 o około 11%. Bez filtrowania przewaga wyniosła odpowiednio około 35% i 17%. Są to względne wzrosty wartości metryk.

Lepszy wynik SASRec wymagał dłuższej nauki: około 24 minut w AWS, wobec około 6 sekund przygotowania MCRS. MCRS szybciej odpowiadał też na pojedyncze zapytanie. Popularity był najszybszy przy rekomendowaniu, ale miał najniższą jakość. Wybór metody zależy więc od tego, czy ważniejsza jest jakość, czy koszt obliczeń.

Większe próbki poprawiły wyniki MCRS i SASRec w wykonanych eksperymentach, ale wydłużyły trening. Na najmniejszej próbce SASRec nie wygrywał we wszystkich metrykach. Pozwalanie na ponowne rekomendacje podnosiło wyniki, ponieważ użytkownicy wracali także do wcześniej oglądanych ofert.

Wyniki dotyczą jednego losowania danych i jednej konfiguracji modeli. Nie sprawdzano ich zmienności dla innych ziaren losowych ani nie wyznaczano przedziałów ufności. Czas zależy od sprzętu i liczby użytkowników obsługiwanych jednocześnie. Ograniczenie danych do popularnych ogłoszeń oznacza też, że badanie nie opisuje jakości rekomendacji dla całego katalogu OLX.

7 Pliki i powtórzenie eksperymentów

Źródła wyników AWS: `metrics.csv`, `dataset.json`, `hardware.json`, `sasrec_losses.csv`, przebieg `20261021T111639_648198Z`. Kod porównania filtrowania i pomiarów: gałąź `SasRec`, commit `aedc612`.

Skrypt dodatkowych próbek: `experiments/run_sizes.py`.
Docelowy katalog oddania: `s3://warsztaty-pw-2026/grupa-5/`.

Literatura

[1] W.-C. Kang, J. McAuley, *Self-Attentive Sequential Recommendation*, ICDM 2018. Implementacja referencyjna: https://github.com/kang205/SASRec.

[2] Repozytorium projektu: https://github.com/als408/Recommendation_systems.